

사용자 경험을 구현하고, 데이터의 흐름을 설계합니다.

React · TypeScript 기반 프론트엔드 개발자

김동영

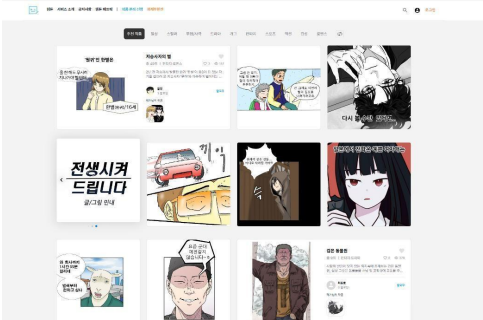
오늘의 웹툰 · Frontend Developer
2021.05 - 2025.02 | 경력 3년 9개월

스타트업의 첫 프론트엔드 개발자로 초기 구축부터 출시·운영까지 참여했습니다. 사용자 흐름을 구현하고, 여러 화면이 공유하는 데이터와 UI의 책임을 정리했습니다.

Dongykim414@gmail.com

010-5357-9878

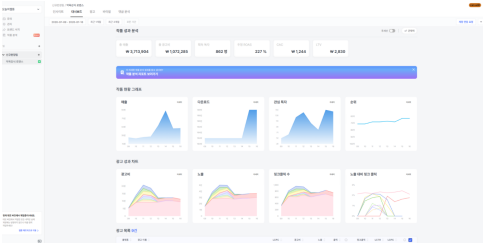
추가 경험: 공통 UI · 운영 도구 · 모바일 출시 → 10



01 오늘의 웹툰

원고 업로드·모바일 피드
공통 데이터 조회와 갱신

CASE STUDIES 02 - 05 →



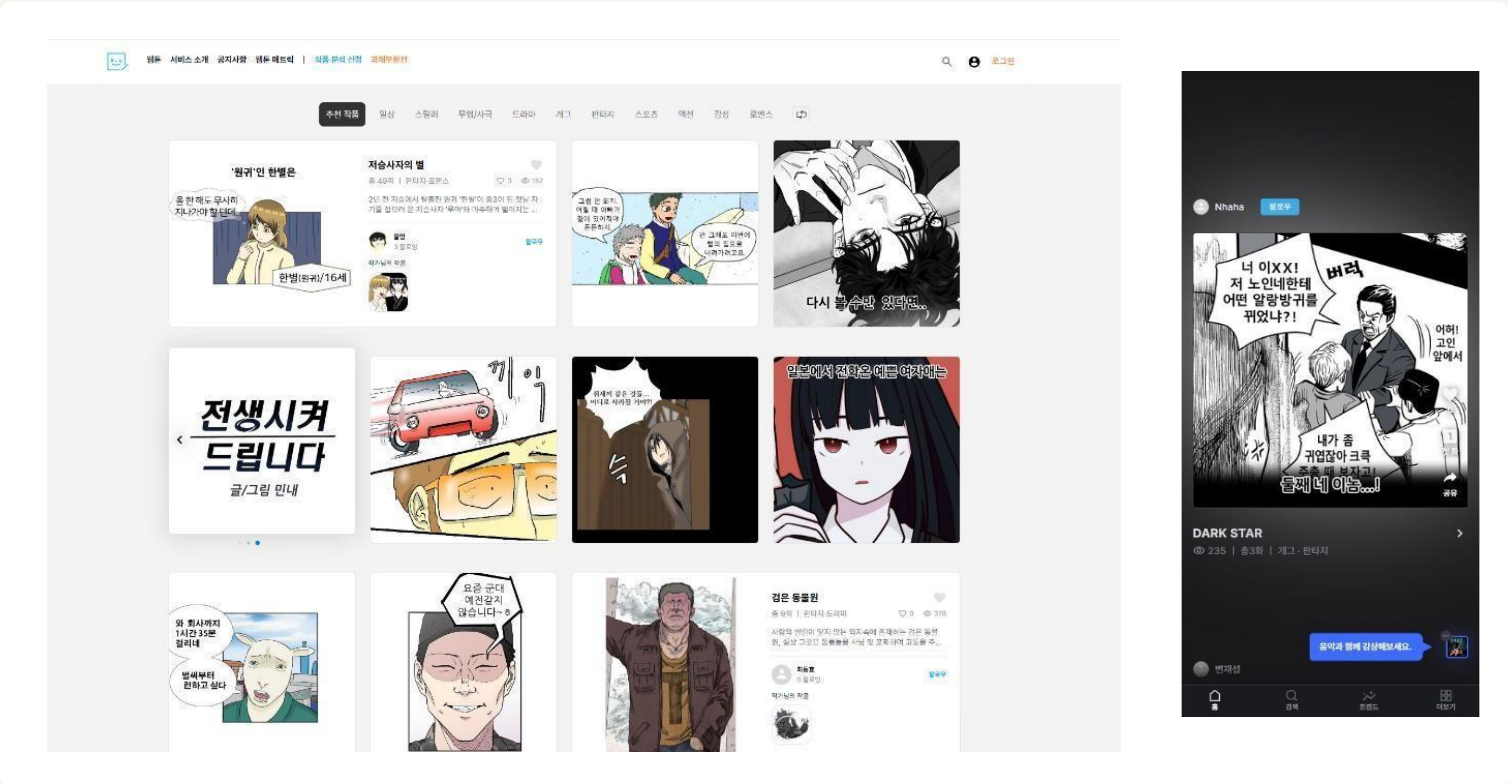
02 웹툰 메트릭

지표 비교·커스텀 차트
고객 시연용 데모·리포트 상태

CASE STUDIES 06 - 09 →

탐색하는 독자와, 원고를 등록하는 작가를 연결했습니다.

오늘의웹툰 | 웹툰 연재 플랫폼



PC 목록 탐색

모바일 세로 피드

실제 서비스 화면 · 기존 포트폴리오 수록 캡처

나의 역할

2022.08 - 2023.09
담당 기능 프론트엔드 단독 개발
백엔드 개발자 1명과 협업

독자의 탐색·감상과 작가의 등록·관리 흐름 중 원고 편집, 모바일 이동, 공통 데이터 조회를 중심으로 개발했습니다.

React · TypeScript · JavaScript
Recoil · Axios · SCSS

challenge.webtoon.today

연결한 사용자 흐름

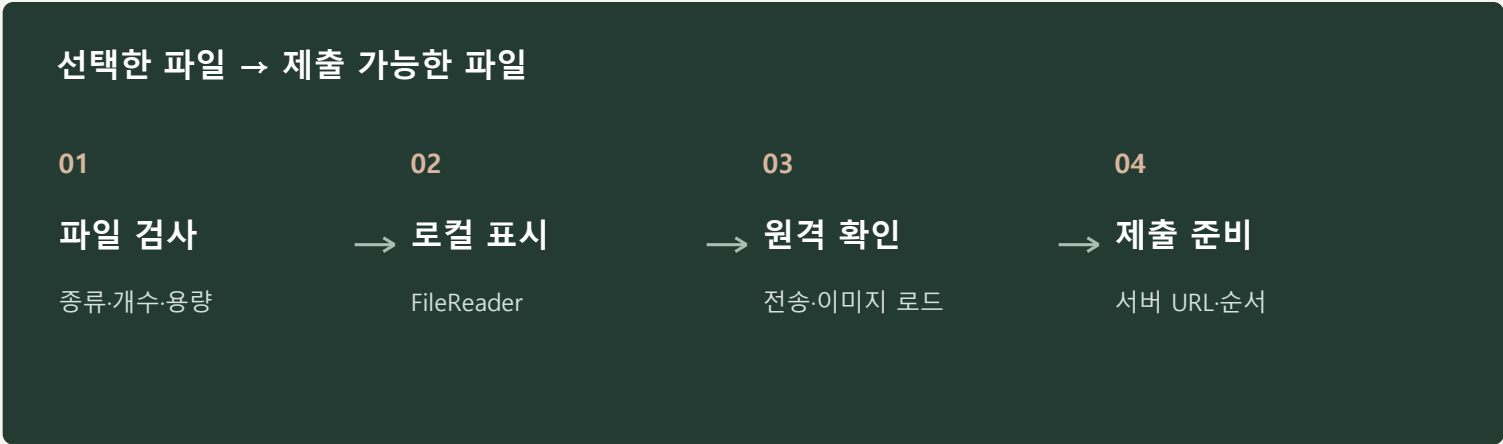
작품·회차 등록 → 원고 확인·편집 → 제출 상태 검증
→ 등록·신청

미리보기와 제출 데이터를 같은 흐름으로 연결했습니다.

핵심 판단: 파일 선택과 업로드 완료는 서로 다른 상태입니다.

사용자가 보는 이미지와 제출되는 URL을 맞추기

파일 읽기와 서버 전송은 끝나는 시점이 다릅니다.
로컬 미리보기를 먼저 표시하고, 업로드한 이미지가
로드되면 서버 URL로 교체하도록 구성했습니다.



구현 결과

파일 추가부터 미리보기·규격 변환·순서 편집을 거쳐
접수 데이터로 전달하는 흐름을 완성했습니다.

용도별 처리

이미지: 원격 이미지 로드 후 미리보기 교체
ZIP: 서버 압축 해제 결과를 편집 목록에 추가
홍보 이미지: Canvas로 600×600 중앙 크롭
단일 썸네일: react-cropper로 수동 크롭

입력별 정렬

마우스는 요소 위치를 비교하고, 터치는 중심 좌표·셀
크기·열 수로 인덱스를 계산했습니다. 업로드 중
정렬과 범위 밖 이동을 제한했습니다.

다음 개선: 안정적인 파일 ID와 파일별 실패·취소·재시도 상태를 도입해 비동기 완료 순서가 달라지는 경우를 검증하겠습니다.

상세 화면에서 돌아와도, 탐색은 이어지도록 했습니다.

핵심 판단: 화면을 넘어 유지할 상태와 이동 중 상태의 수명을 구분했습니다.



바깥 피드는 직접 제어

touch·mouse·wheel 입력과 카드 크기·이동 방향·임계값으로 목표 인덱스를 계산하고 transform·transition으로 스냅했습니다. wheel에는 짧은 입력 제한을 두었습니다.

카드 내부는 Swiper

이미지 페이드·자동 재생은 Swiper를 사용했습니다. 재생 완료 콜백과 바깥 피드 이동을 연결하고, 전환 후 마지막 몇 장에 접근하면 추가 데이터를 요청했습니다.

직접 제어의 비용

카드 경계·타이머 정리는 직접 관리합니다. 다음에는 저장된 작품이 목록에 없는 경우와 애니메이션 중 새 입력을 우선 검증하겠습니다.

상태의 저장 위치

Recoil

마지막 작품 키 · 앱 내 재진입 복원

React state

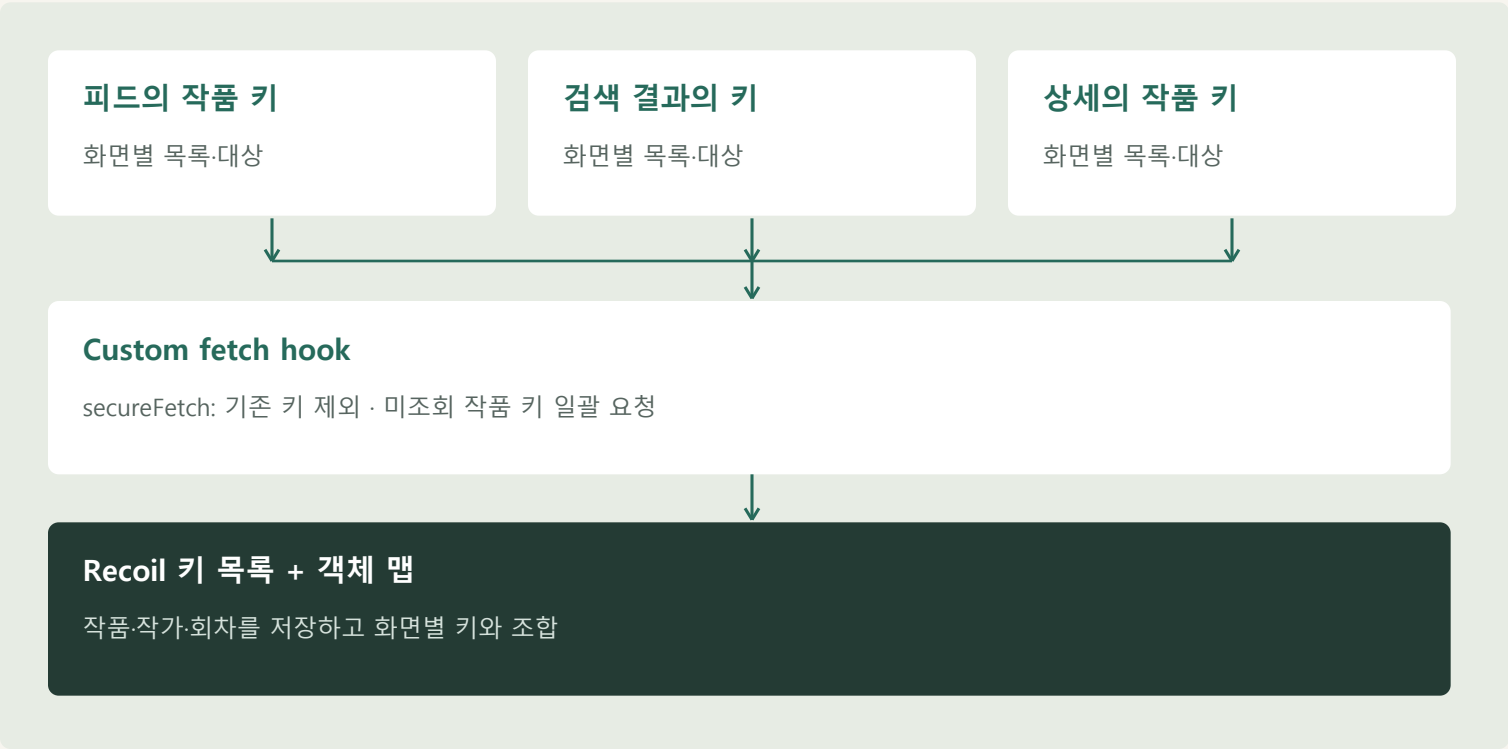
현재 포커스 작품 · 화면 반영

ref

드래그 여부·좌표 · 이벤트 간 추적

화면은 달라도, 작품 데이터는 함께 사용합니다.

핵심 판단: 화면별 목록의 키와 공통 데이터 객체를 분리했습니다.



함께 연결한 접수 흐름

인증·필수 정보·업로드 상태 검증 → 작품 → 회차 → 콘텐츠 → 신청 API 순차 호출. 단계별 실패 안내와 작품 소유권에 따른 감상·관리 UI를 연결했습니다.

조회와 갱신을 별도 경로로

secureFetch는 기존 키를 걸러 필요한 데이터만 조회합니다. 작품 다중 키는 한 API 요청에 묶었습니다.

refreshTitle·refreshUser·회차 refresh는 변경된 데이터를 명시적으로 다시 가져오는 경로로 두었습니다.

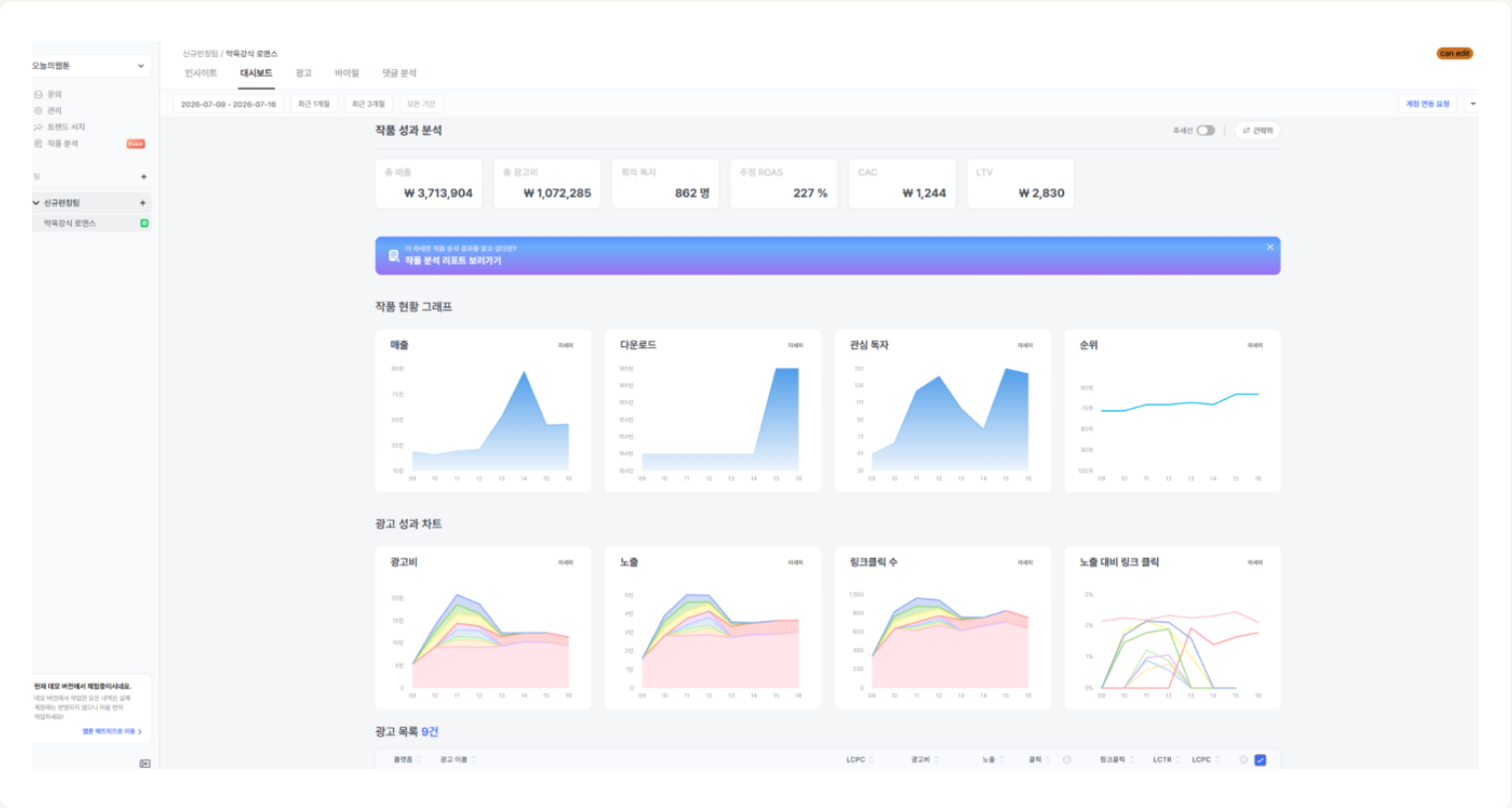
구현 결과

피드·검색·상세가 공통 작품·작가 데이터를 재사용하고, 화면은 공통 혹은 조회·갱신을 연결했습니다.

다음 개선: 요청 상태와 성공한 캐시를 분리하고 재시도·무효화 정책을 명시하겠습니다.

성과 수치를, 고객이 비교할 수 있는 리포트로 만들었습니다.

웹툰 메트릭 | B2B 데이터 리포트·시각화



요약 지표 → 작품·광고 추이 → 상세 분석

실제 서비스 데모 화면 · 기존 포트폴리오 수록 캡처

나의 역할

2023.10 - 2024.09

Frontend Developer

FE 2명 / BE 2명

API 응답을 화면용 데이터로 가공하고, 커스텀 차트·유료 리포트 상태·고객 시연용 데모 환경을 구현했습니다.

React · TypeScript · D3.js

SVG · Axios · visx

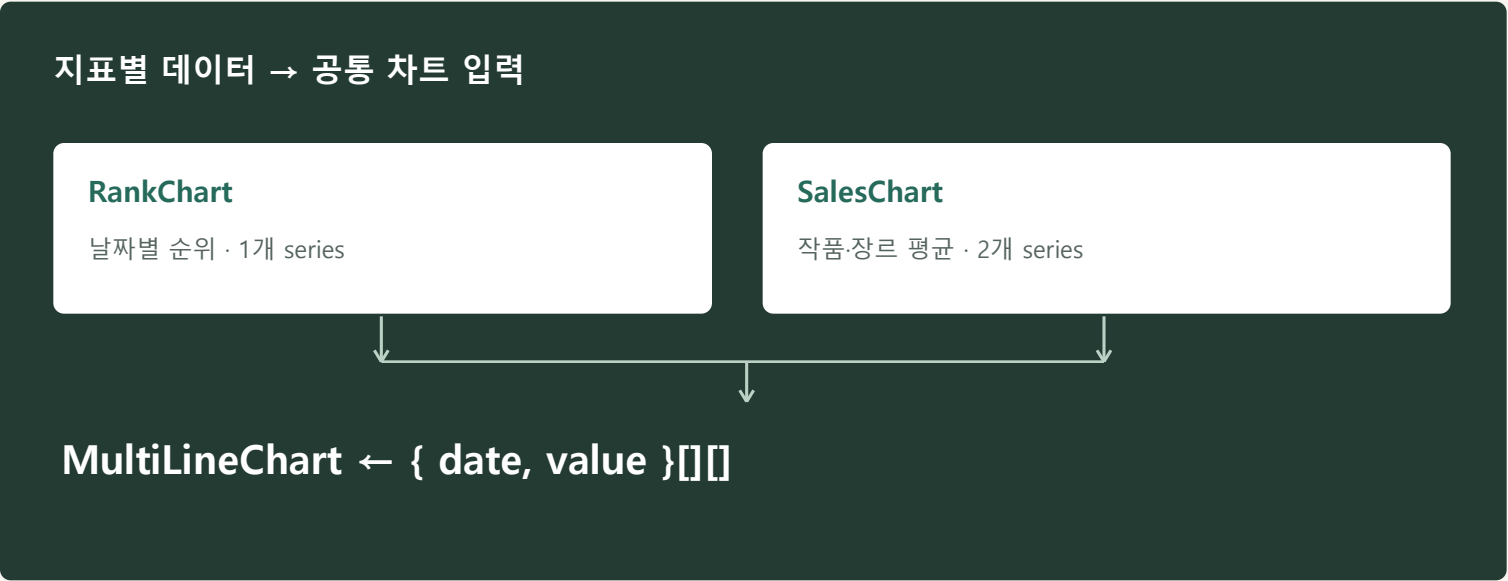
metric.webtoon.today

고객에게 필요한 기준

작품의 추이, 장르 평균과의 차이, 각 지표의 의미를 함께 설명했습니다.

지표의 의미는 남기고, 차트의 표현은 재사용했습니다.

핵심 판단: 데이터 가공은 지표별 컴포넌트가, 계산은 D3가, SVG 표현은 React가 맡습니다.



공통화하면서 유지한 의미

순위는 낮은 숫자가 위에 오도록 Y축을 반전했습니다.
날짜·숫자 축을 구분하고 여백·색상·영역·격자 표시를 props로 조합했습니다.

대상 작품·장르 평균을 비교하고 단위·범례·요약을 연결했습니다. 차트에 수치와 비교 기준을 함께 담았습니다.

D3: 수치와 경로 계산

extent · scaleLinear · scaleTime
line · area · tick · 좌표 계산

React: 계산 결과를 SVG로

path · line · text · circle
격자·레이블·영역·툴팁 표시

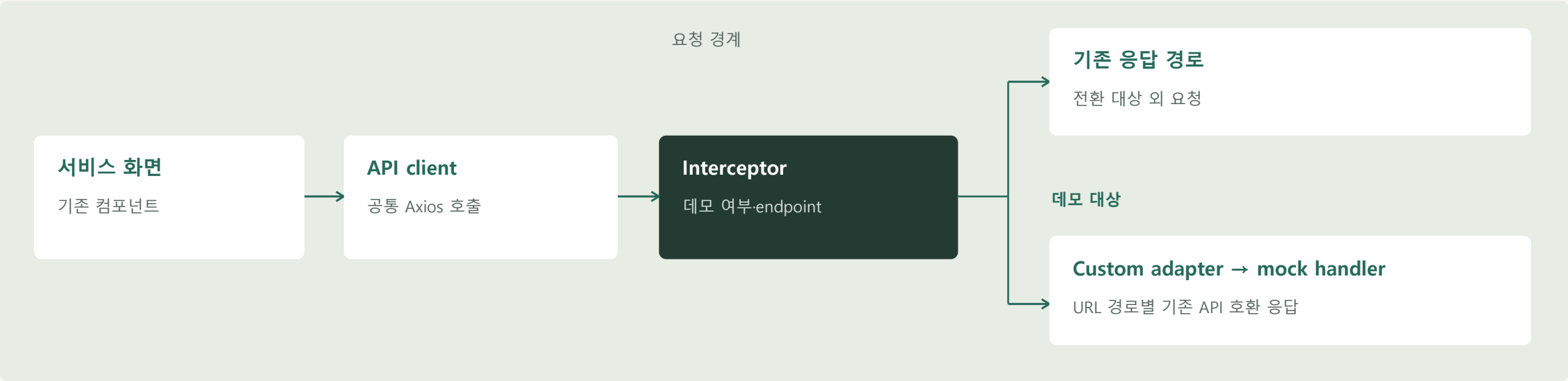
구현 결과

순위·매출이 같은 선형 차트를 사용하도록 구성하고, 지표별 표현 차이를 공통 컴포넌트의 설정으로 수용했습니다.

설계 비용: 빈 데이터·좁은 값 범위·축 눈금·툴팁 좌표 같은 경계 조건을 직접 관리해야 합니다.

같은 화면에서, 응답 경로를 바꿔 고객 시연을 만들었습니다.

핵심 판단: 운영 UI와 API 호출 구조를 공유하고 Axios 요청 경계에 전환 지점을 뒀습니다.



전환 로직

데모 진입 시 모드 설정과 세션 갱신을 연결했습니다. 인터셉터에서 요청을 분류하고, URL 경로에 맞는 mock handler를 선택했습니다.

기존 화면을 그대로 활용

mock handler가 기존 API와 호환되는 응답을 반환하도록 구성했습니다. 기존 화면에서 고객 시연과 후속 상담에 활용했습니다.

지원 범위와 유지보수

미지원 기능은 안내와 오류 응답으로 처리했습니다. API 형식이나 기능이 바뀌면 mock handler와 지원 범위도 함께 관리해야 합니다.

같은 리포트도, 준비 상태에 따라 다른 정보를 보여줍니다.

핵심 판단: 리포트의 구성은 유지하고 상세 데이터와 안내를 상태별로 분기했습니다.

Pending	Generating	Complete
구매 대기	생성 중	생성 완료
마스킹된 수치	생성 중 안내	준비된 상세 수치
프리뷰 차트	준비되지 않은	분석 차트
구매 안내	상세 데이터 표시 제한	상세 리포트 제공

프론트엔드의 담당 범위

서버가 제공한 상태와 데이터에 맞춰 마스킹 값, 프리뷰 차트, 상세 수치와 안내 UI를 연결했습니다. 데이터가 없는 차트에는 별도 프리뷰 표현을 제공했습니다.

권한과 결제의 책임

유료 데이터의 제공 권한과 결제 처리는 서버의 책임으로 구분했습니다. 화면의 마스킹은 현재 상태를 표현하는 UI로 구성했습니다.

공통 UI를 만들고, 제품을 출시·운영했습니다.

대표 프로젝트에서 다룬 책임 분리와 재사용을 다른 개발 경험으로 확장했습니다.

01 공통 UI

Toast·Loading·Drawer를 TypeScript 컴포넌트와 Hook으로 구축해 3개 서비스에 적용했습니다. Context로 열린 Drawer의 순서를 공유하고 최상단만 Escape를 처리하도록 구성했습니다.

React · TypeScript · Context API · React Portal

02 운영 도구·데이터 표현

한글 초성·불완전 음절 검색과 미조회 작품 일괄 요청을 구현했습니다. 집계 데이터를 Chart.js 구조로 변환하고 도달·열람·참여·독자 특성 순으로 정보를 재구성했습니다.

React · Redux · Axios · Chart.js

03 iOS·Android 출시

Unity에서 JSON 기반 스토리·대사·캐릭터 연출과 주요 클라이언트 기능을 구현했습니다. 기기 테스트와 App Store·Google Play 심사 대응부터 출시까지 담당했습니다.

04 협업과 운영

PR 리뷰에서 컴포넌트 책임 분리와 유지보수를 논의하고 인턴의 온보딩 문서·초기 과제·코드 리뷰를 지원했습니다. Amplitude와 CloudWatch로 운영 이슈의 원인을 추적했습니다.

김동영 | Frontend Developer

Dongykim414@gmail.com · 010-5357-9878